

ADVANTAGEKIT

Theory

原理

中英对照 · 9 篇文章

按照 AdvantageKit 文档顺序编排

Replay Case Studies / 回放案例

Contents / 目录

01 Log Replay Comparison / 日志回放对比	3
02 High-Frequency Odometry / 高频 odometry	23
03 Deterministic Timestamps / 确定性时间戳	28
04 Elevator Profile / 升降机构运动曲线	30
05 Autoscoring / 自动得分	30
06 Command Gremlins / Command 中的隐蔽问题	30
07 Aiming Functions / 瞄准函数	30
08 AprilTag Vision / AprilTag 视觉	30
09 Traditional Vision / 传统视觉	30

Log Replay Comparison / 日志回放对比

FRC teams have access to multiple logging tools that feature "replay" capabilities. These fall into the categories of **deterministic replay** (AdvantageKit) and **nondeterministic replay** (Hoot Replay). Each type of replay framework offers significantly different capabilities with regard to determinism, playback functionality, and code structure. This page compares these tools to help teams understand their key differences.

FRC 队伍可以使用多种具有“回放”功能的日志工具。它们可分为**确定性回放** (AdvantageKit) 和**非确定性回放** (Hoot Replay) 两类。各类回放框架在确定性、回放功能和代码结构方面提供的能力有显著差异。本页对这些工具进行比较，帮助队伍理解它们的主要区别。

NOTE / 说明

Many non-replay logging options are also available (such as [WPILib data logging](#) and [Epilogue](#)), but this page focuses exclusively on replay-compatible logging tools.

还有许多不支持回放的日志方案（例如 [WPILib 数据日志](#) 和 [Epilogue](#)），但本页只讨论支持回放的日志工具。

Determinism / 确定性

The biggest difference between replay frameworks is the ability of each tool to replay robot code logic in a way that is **consistent, trustworthy, and robust to timing inconsistency**.

回放框架之间最大的区别，在于各工具能否以**一致、可信且能抵御时序不一致影响**的方式回放机器人代码逻辑。

Deterministic (AdvantageKit)	Non-Deterministic (Hoot Replay)
确定性 (AdvantageKit)	非确定性 (Hoot Replay)
The replayed robot code will always match the behavior of the real robot. The results of replay can be trusted completely to match the actual behavior of the robot. 回放的机器人代码始终与真实机器人的行为一致。可以完全信任回放结果与机器人的实际行为相符。	No guarantees are made about the accuracy of replay in simulation. Data may arrive in replay at different times or at different rates than the real robot, which impacts the accuracy of all parts of the robot code. 不保证仿真中回放的准确性。与真实机器人相比，数据在回放中可能在不同时间到达，或以不同速率到达，这会影响机器人代码所有部分的准确性。

Determinism has a major impact on the practicality of log replay, since running simulation **faster than real-time** is a core part of the debugging process in practice. The accuracy of deterministic replay is unaffected by replay speed, while the accuracy of non-deterministic replay decreases when running at faster rates.

确定性对日志回放的实用性有重大影响，因为在实际调试过程中，以**快于实时的速度**运行仿真是核心环节。确定性回放的准确性不受回放速度影响，而非确定性回放在运行速度更快时，准确性会下降。

Why Does It Matter? / 为什么这很重要？

We are often asked by teams why they should care about deterministic replay. Non-deterministic replay creates butterfly effects that severely impact the accuracy of replay.

队伍经常问我们，为什么应该重视确定性回放。非确定性回放会产生蝴蝶效应，严重影响回放的准确性。

The Butterfly Effect / 蝴蝶效应

The **butterfly effect** describes how small differences in the inputs to a complex system (like robot code) have ripple effects that can significantly impact the system's behavior in the future. Minor differences in inputs can have a much larger effect on outputs than one might intuitively expect.

蝴蝶效应描述的是：复杂系统（如机器人代码）的输入出现微小差异时，会产生连锁影响，显著改变系统未来的行为。输入的细微差异对输出的影响，可能远大于直觉预期。

The sequence below provides a simple example of how non-deterministic inputs can impact important parts of the robot code:

以下过程给出了一个简单示例，说明非确定性输入如何影响机器人代码中的重要部分：

1. A vision measurement from a camera is lost or delayed due to non-deterministic replay.
由于非确定性回放，来自相机的一次视觉测量丢失或延迟。
2. When combined with odometry data in a pose estimator, the estimated pose of the robot is incorrect for one or more loop cycles.
在位姿 (pose) 估计器中与里程计 (odometry) 数据融合后，机器人在一个或多个循环周期中的估计 pose 出现错误。
3. An auto-align command waits for the robot to be within tolerance before scoring. This is a precise operation where errors of less than a centimeter can have a major impact.
一个自动对齐 Command 会等待机器人进入容差范围后再得分。这是一项精密操作，小于一厘米的误差也可能造成重大影响。
4. The driver presses a button to score just after the real robot is within tolerance. Since the replayed robot's pose is inaccurate, the auto-score command rejects the button input in replay (even though it was accepted on the real robot).
驾驶员在真实机器人刚进入容差范围后按下得分按钮。由于回放机器人的 pose 不准确，自动得分 Command 在回放中拒绝了该按钮输入（尽管真实机器人接受了它）。
5. The superstructure of the robot is now being commanded to a different state on the real robot and in replay, since only the real robot continues the scoring operation.
由于只有真实机器人继续执行得分操作，真实机器人和回放中对 superstructure 发出的状态指令现在不同。
6. Setpoints to individual mechanisms are now drastically different between the real robot and replay, and do not match the inputs (e.g. encoders) provided to replay. Any tolerance checking of mechanisms is likely to be nonfunctional for the rest of the replay.
真实机器人和回放中发给各机构的 setpoint 现在存在巨大差异，也与提供给回放的输入（例如编码器）不匹配。在回放的剩余过程中，机构的任何容差检查都很可能无法正常工作。
7. Future control inputs will not be correctly obeyed in replay, since the states of many commands and subsystems no longer match the real robot.
后续控制输入将无法在回放中得到正确执行，因为许多 Command 和 subsystem 的状态已经与真实机器人不同。

This scenario may seem specific, but similar divergences are **almost inevitable** when replaying robot code of moderate complexity. The testing in the next section demonstrates what this effect looks like in practice.

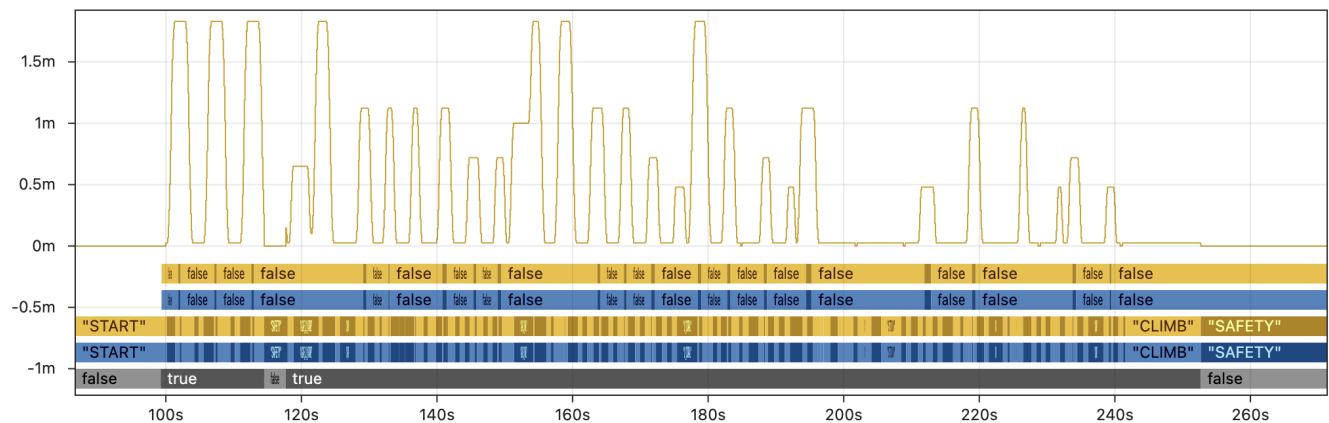
这一情景看似具体，但在回放中等复杂度的机器人代码时，类似的偏离**几乎不可避免**。下一节的测试展示了这种效应在实际中的表现。

To demonstrate the impact of deterministic replay, the graphics below show real log data from Team 6328's 2025 robot. To represent each category of replay, this data is based on AdvantageKit's deterministic replay (running ~50x faster than real-time) and a close approximation of Hoot Replay (running <5x faster than real-time).

为展示确定性回放的影响，下方图表使用了 6328 队 2025 年机器人的真实日志数据。为了代表各类回放，数据基于 AdvantageKit 的确定性回放（以约 50x 实时速度运行）以及对 Hoot Replay 的近似实现（以 <5x 实时速度运行）。

First, the image below shows a few key fields from the AdvantageKit replay. Outputs from the real robot are in blue and outputs from AdvantageKit replay are in yellow. The line graph shows the commanded setpoint of the elevator. From bottom to top, the discrete fields show the enabled state, superstructure state, and whether the robot is in tolerance for scoring. Every field displayed here is an **exact match between real and replay**, providing complete trust in the accuracy of the data.

首先，下图显示了 AdvantageKit 回放中的几个关键字段。真实机器人的输出为蓝色，AdvantageKit 回放的输出为黄色。折线图显示升降机构被指令设定的 setpoint。离散字段从下到上分别表示启用状态、superstructure 状态，以及机器人是否处于得分所需的容差范围内。这里显示的每个字段在**真实运行与回放之间都完全一致**，因此可以完全信任数据的准确性。



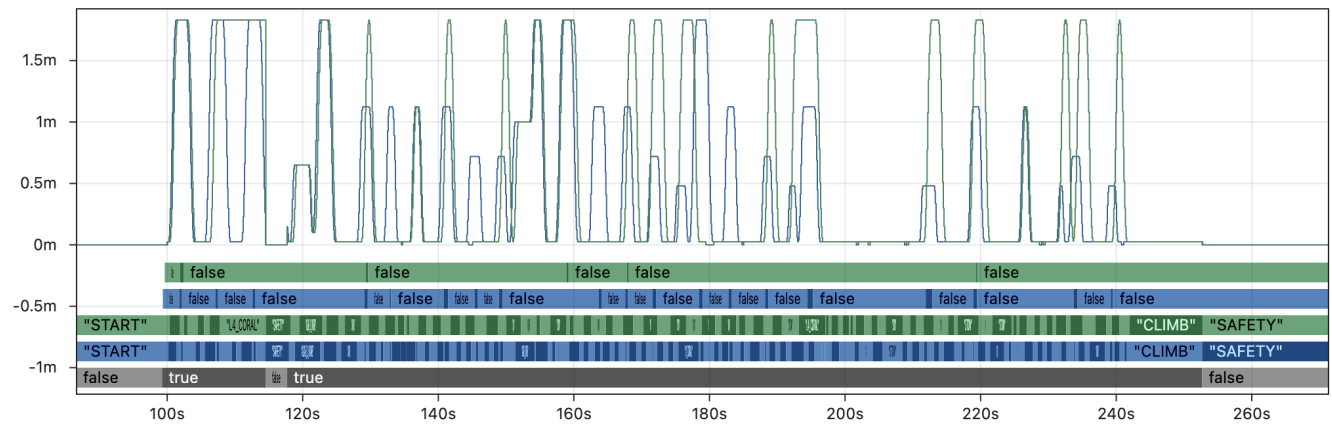
AdvantageKit Replay / AdvantageKit 回放

By contrast, the image below shows the same fields with a close approximation of Hoot Replay running 5x faster than real-time. This is still about 10x slower than AdvantageKit and largely impractical for real debugging workflows. The example shown here is also a best case scenario which includes extensive modifications to the code that compensate for the difference in replay speed.

相比之下，下图显示了以 5x 实时速度运行的 Hoot Replay 近似实现中的相同字段。这仍比 AdvantageKit 约慢 10x，基本无法满足实际调试工作流程的需要。这里展示的示例还是一种最佳情况，其中对代码进行了大量修改，以补偿回放速度的差异。

Outputs from the real robot are in blue and outputs from an approximation of Hoot Replay are in green. Within a few seconds of starting the autonomous routine, the state of the robot has **completely diverged between real and replay** due to the butterfly effect. This significantly reduces the value of the log data for debugging, as it no longer resembles the original behavior.

真实机器人的输出为蓝色，Hoot Replay 近似实现的输出为绿色。自动程序开始后仅几秒钟，由于蝴蝶效应，机器人状态就在**真实运行与回放之间完全偏离**。这显著降低了日志数据的调试价值，因为它已不再反映原始行为。



Hoot Replay (5x, modified) / Hoot Replay (5x, 已修改)

Keep in mind that replayed outputs are most useful when the equivalent values were not recorded by the real robot (i.e. there is no reference point to verify accuracy). For that critical use case, there is no way to distinguish accurate outputs from the inaccurate, diverged outputs shown above. This undermines the core purpose of replay, as the outputs cannot be trusted for debugging.

请记住，当真实机器人未记录对应值时，回放输出才最有用（即没有可用于验证准确性的参考值）。对于这一关键用途，无法将准确输出与上图中不准确、已偏离的输出区分开来。这削弱了回放的核心作用，因为调试时无法信任这些输出。

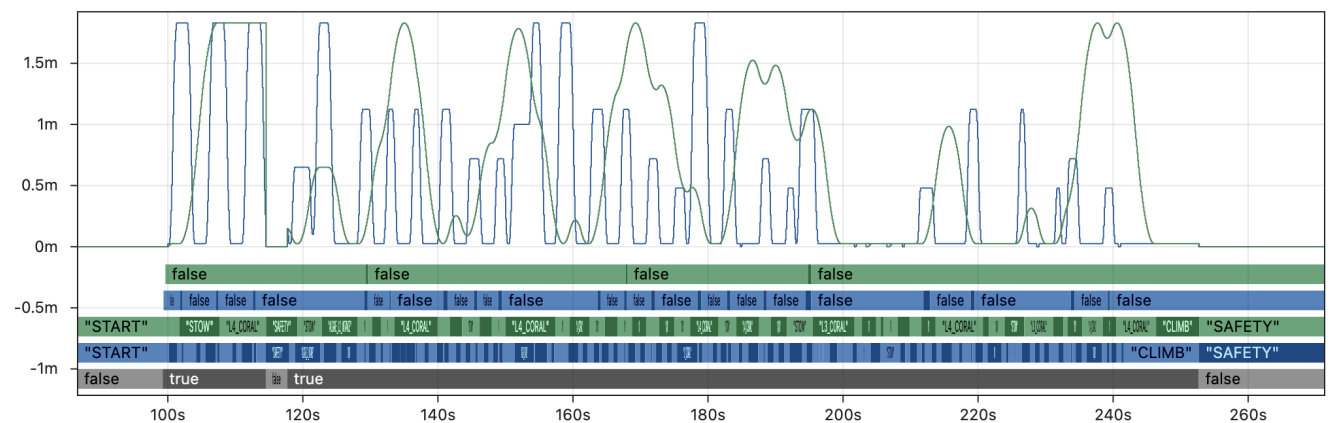
More Details / 更多细节

The graphs above shows the results of replaying 5x faster than real-time with additional modifications to compensate for loop cycle time, though these changes would not be part of a typical robot project. We have provided several other test cases to demonstrate the impact of different replay settings:

上方图表显示的是以 5x 实时速度回放的结果，并额外修改了代码以补偿循环周期时间；这些修改通常不会出现在一般机器人项目中。我们还提供了其他几个测试案例，以展示不同回放设置的影响：

5x faster than real-time, typical robot project:

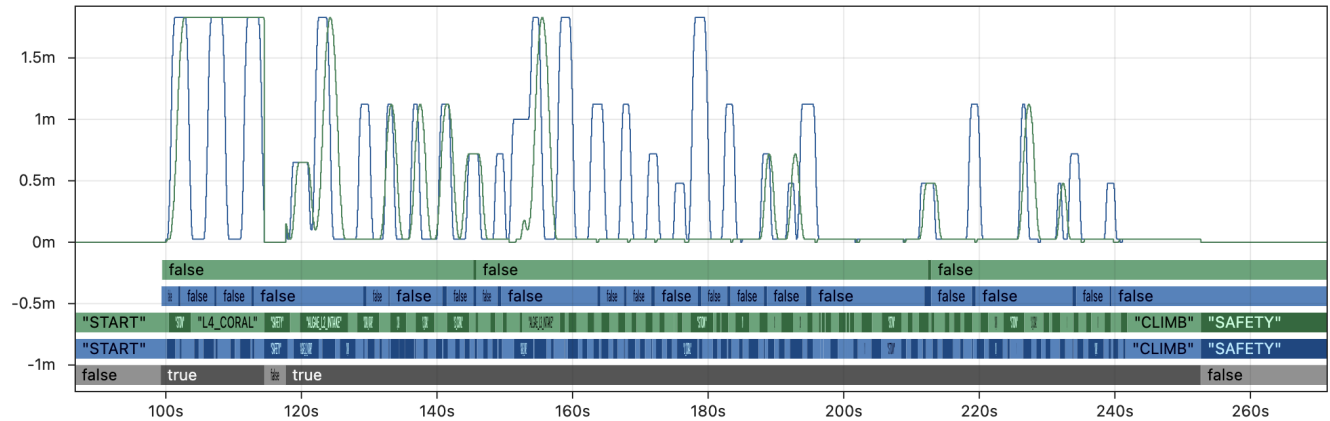
5x 实时速度，常规机器人项目：



Hoot Replay (5x, unmodified) / Hoot Replay (5x, 未修改)

2x faster than real-time, typical robot project:

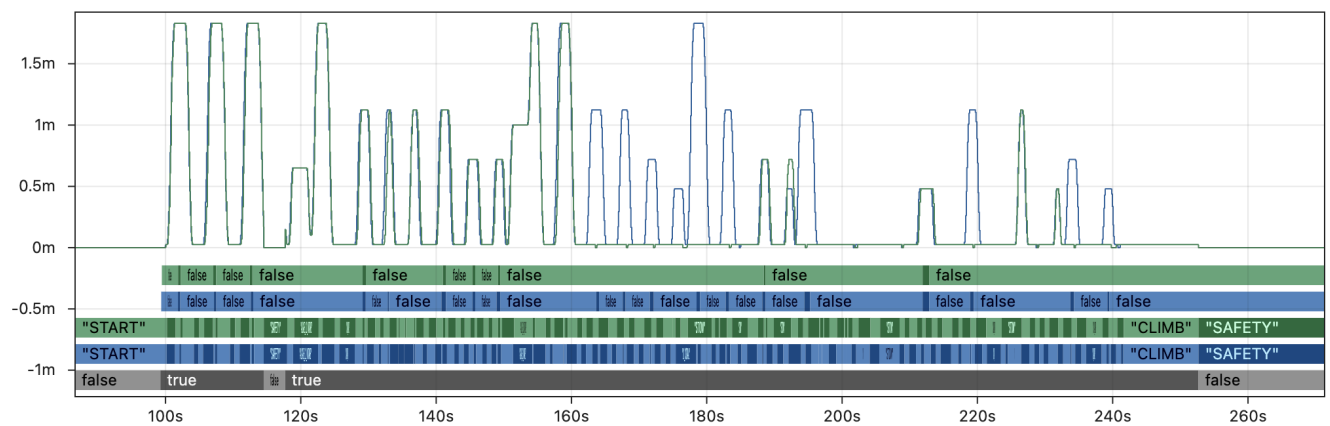
2x 实时速度，常规机器人项目：



Hoot Replay (2x, unmodified) / Hoot Replay (2x, 未修改)

2x faster than real-time, compensated:

2x 实时速度，已补偿：



Hoot Replay (2x, modified) / Hoot Replay (2x, 已修改)

Note that even the very best case shown in the last graph still breaks down completely midway through the match, and is unable to replay critical fields like the auto scoring tolerance.

请注意，即使最后一张图展示的最理想情况，仍然会在比赛中途完全失去一致性，无法回放自动得分容差等关键字段。

What about other fields?

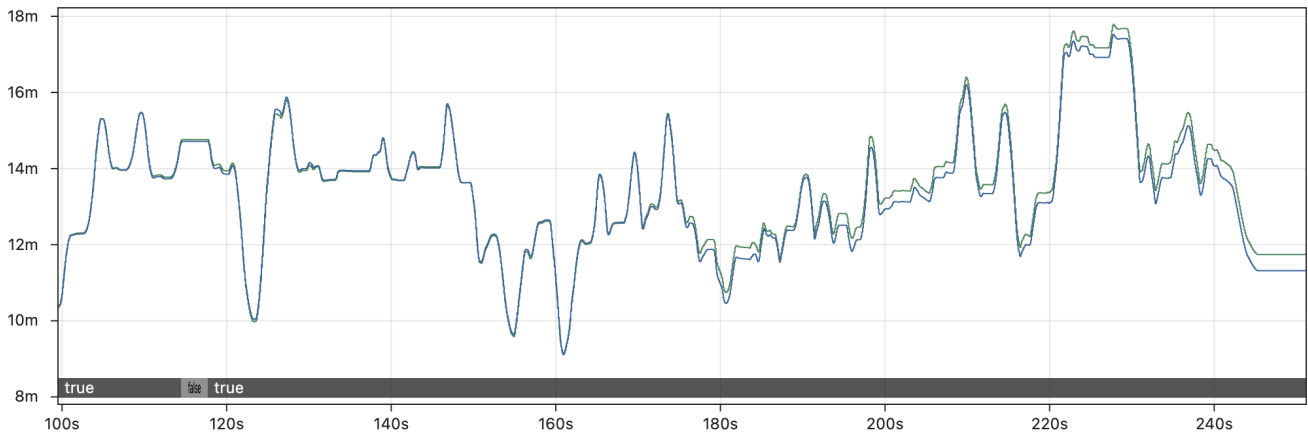
其他字段呢？

It is true that some fields are more affected by replay inaccuracy than others. For example, the graph below compares the X position of drive odometry between the real robot and Hoot Replay running 5x faster than real-time. Odometry is only affected by the drive motors, so it is less subject to the butterfly effect than other parts of the code (though it still drifts several feet by the end of the match).

确实，有些字段比其他字段更容易受到回放不准确的影响。例如，下图比较了真实机器人与以 5x 实时速度运行的 Hoot Replay 中，底盘 odometry 的 X 位置。odometry 只受底盘电机影响，因此比代码的其他部分更不容易受到蝴蝶效应影响（不过，到比赛结束时仍会漂移数英尺）。

Log replay is most helpful when untangling complex code logic that is nontrivial to recreate without the full set of input data, as demonstrated even in our [simplest examples](#). Odometry data and other trivial fields serve as a partial exception to the butterfly effect, but (as noted above) the lack of reference points when running replay in practice means that it is never possible to distinguish non-deterministic outputs that are slightly inaccurate (odometry) from the majority of outputs that are completely inaccurate.

日志回放在梳理复杂代码逻辑时最有帮助；如果没有完整的输入数据，这些逻辑很难重现，即使我们[最简单的示例](#)也展示了这一点。odometry 数据和其他简单字段可在一定程度上成为蝴蝶效应的例外，但如上所述，实际运行回放时缺乏参考值，意味着始终无法将 略有误差 的非确定性输出 (odometry) 与占多数的 完全不准确 的输出区分开来。



Odometry: Hoot Replay (5x, modified) / Odometry: Hoot Replay (5x, 已修改)

What about skipping in time?

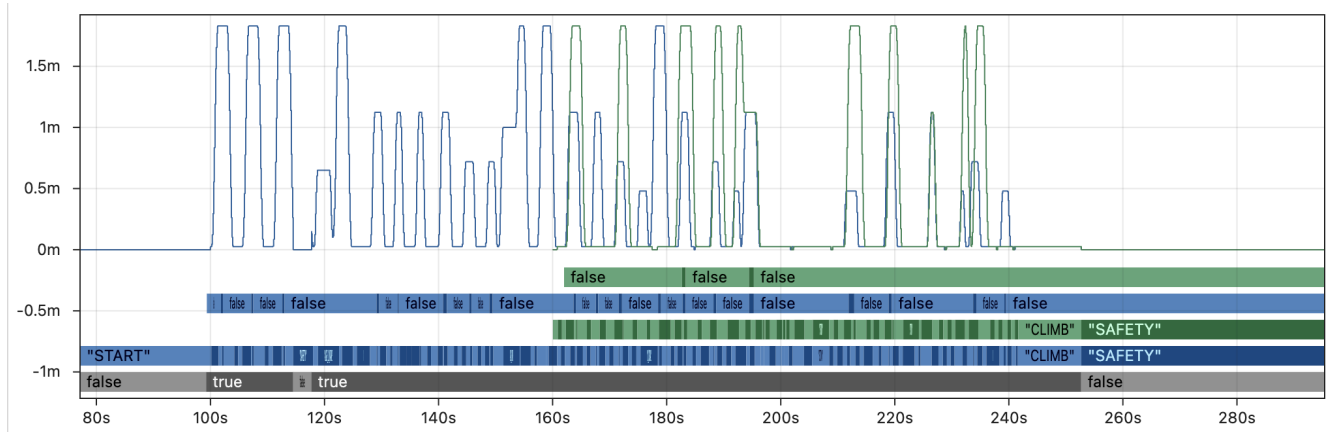
在时间上跳过一段呢？

The [section below](#) explains why rapid iteration and running faster than real-time are critical to any replay workflow, which is why the examples above demonstrate the impact of running Hoot Replay faster than real-time. However, one could also start the replay at a later point in the log file to work around the slow speed of non-deterministic replay.

[下方章节](#)解释了为什么快速迭代和以快于实时的速度运行对任何回放工作流程都至关重要，这也是上面的示例展示 Hoot Replay 以快于实时的速度运行所产生影响的原因。不过，也可以从日志文件中较晚的位置开始回放，以绕过非确定性回放速度缓慢的问题。

The graph below demonstrates why this approach is ineffective, by skipping to the middle of teleop before running simulated Hoot Replay (2x faster than real-time with loop cycle compensation). Even in this best-case scenario for Hoot Replay running at only 2x speed, the replay is completely unable to match the real outputs. Skipping large parts of the log massively increases the impact of the butterfly effect by completely changing the set of inputs accessible to the replayed code. One should not expect to see accurate outputs at any speed unless all of the inputs are accounted for during replay.

下图通过跳到 teleop 中途，再运行模拟的 Hoot Replay (2x 实时速度，并补偿循环周期)，展示了为什么这种方法无效。即使这是 Hoot Replay 仅以 2x 速度运行的最佳情况，回放仍完全无法与真实输出一致。跳过日志的大段内容会完全改变回放代码可访问的输入集合，从而大幅放大蝴蝶效应的影响。除非在回放中完整考虑所有输入，否则不应期待在任何速度下获得准确输出。



Skipping: Hoot Replay (2x, modified) / 跳过: Hoot Replay (2x, 已修改)

Rapid Iteration / 快速迭代

Log replay can be used in a variety of environments, which take advantage of the ability to rapidly iterate on code or debug issues without access to the robot. Here are a few examples where replay can play a critical role in the debugging process:

日志回放可用于多种环境，利用它可以快速迭代代码，或在无法接触机器人时调试问题。以下是回放能够在调试过程中发挥关键作用的几个例子：

- Debugging complex logic issues between matches without access to a practice field.
在比赛间隙、无法使用练习场时，调试复杂的逻辑问题。
- Retuning an auto-score tolerance in the pits based on data from the last match.
在维修区根据上一场比赛的数据，重新调整自动得分容差。
- Testing a variety of vision filtering techniques between in-person meetings.
在两次线下会面之间，测试多种视觉过滤技术。
- Remotely debugging issues for a team by repeatedly logging additional outputs.
通过反复记录额外输出，远程帮助队伍调试问题。
- Generating outputs after every match that are too complex to run on the RoboRIO.
每场比赛后，生成复杂到无法在 RoboRIO 上运行的输出。

Every one of these use cases **depends on being able to run replay faster than real-time**. A typical match log may be 10 minutes long, and a replay feature that takes 10 minutes to run is not practical in any of these scenarios. Whether log replay is used under time pressure at an event or at home for rapid debugging, quickly running multiple replays with different outputs or tuning parameters is absolutely core to its utility.

这些用途中的每一项都**依赖于以快于实时的速度运行回放的能力**。一份典型的比赛日志可能长达 10 分钟；在这些场景中，耗时 10 分钟的回放功能都不实用。无论是在赛事中有时间压力时使用日志回放，还是在家中快速调试，使用不同输出或调参值快速运行多次回放，都是其价值的绝对核心。

Comparison / 对比

AdvantageKit	Hoot Replay
[+] Run as fast as possible (e.g. ~50x real-time)	[-] Accuracy decreases with faster speeds
[+] 尽可能快地运行（例如约 50x 实时速度）	[-] 速度越快，准确性越低
[+] Replay Watch for fast iteration	[-] Replay process is fully manual
[+] 使用 Replay Watch 快速迭代	[-] 回放过程完全依靠手动操作
[+] Pull and push logs directly to AdvantageScope	[-] Manual file management
[+] 直接从 AdvantageScope 拉取日志或将日志推送到 AdvantageScope	[-] 手动管理文件

Deterministic replay means that accuracy is unaffected by the replay speed. Running replay ~50 times faster than real-time is common, which means that **a 10 minute match log can be replayed in just 12 seconds**. AdvantageKit is designed to make rapid iteration as painless as possible through features like [Replay Watch](#) and integration with AdvantageScope; just open a log, run replay, and see the results with no manual log management required.

确定性回放意味着准确性不受回放速度影响。以约 50 倍实时速度运行回放很常见，这意味着**一份 10 分钟的比赛日志，仅需 12 秒 就能完成回放**。AdvantageKit 通过 [Replay Watch](#) 以及与 AdvantageScope 集成等功能，尽量降低快速迭代的操作负担；只需打开日志、运行回放并查看结果，无需手动管理日志。

By contrast, Hoot Replay's non-deterministic approach presents users with difficult trade-offs between accuracy and practicality. Running at just 5x speed already has a **major impact on accuracy while still taking a full 2 minutes** per replay iteration. Non-determinism makes replay more difficult to use in the high-pressure scenarios where it matters the most.

相比之下，Hoot Replay 的非确定性方案使用户不得不在准确性与实用性之间艰难权衡。仅以 5x 速度运行，就已经**对准确性产生重大影响，同时每次回放迭代仍需完整的 2 分钟**。非确定性使回放在最需要它的高压场景中更难使用。

The video below demonstrates what the difference in speed between deterministic and non-deterministic replay looks like in practice on a short 5:48 match log. Several replays of the same log are synchronized and shown in real-time.

下方视频通过一份较短的 5:48 比赛日志，展示了确定性与非确定性回放之间的速度差异在实际中的表现。同一日志的多次回放被同步，并以实时速度展示。

[Log Replay Speed Comparison](#)

[日志回放速度对比](#)

Code Structure / 代码结构

While Hoot Replay involves significant trade-offs, its core design goal is to "simplify" hardware interactions. Unlike AdvantageKit, some subsystems may be compatible with Hoot Replay while using CTRE's standard subsystem structure (combining high-level logic, hardware configuration, low-level controls, and simulation in a single class).

虽然 Hoot Replay 涉及显著的权衡，但其核心设计目标是“简化”硬件交互。与 AdvantageKit 不同，有些 subsystem 在采用 CTRE 标准的 subsystem 结构（将高层逻辑、硬件配置、底层控制和仿真合并在一个类中）时，可能仍兼容 Hoot Replay。

Subsystems under Hoot Replay fall into the two categories shown below. For subsystems that are not natively compatible, **every input must be manually logged and replayed**.

Hoot Replay 中的 subsystem 分为以下两类。对于原生不兼容的 subsystem，**每个输入都必须手动记录和回放**。

Natively Compatible 原生兼容	Manual Logging 手动记录日志
Phoenix 6 CTRE devices Phoenix 6 CTRE 设备	- All other CTRE devices 所有其他 CTRE 设备 - Non-CTRE devices 非 CTRE 设备 - Non-CAN sensors (e.g. analog encoders) 非 CAN 传感器（例如模拟编码器） - Network devices (e.g. Limelight, PhotonVision) 网络设备（例如 Limelight、PhotonVision） - Dashboard inputs (e.g. auto choosers) Dashboard 输入（例如自动程序选择器）

Hardware Abstraction vs. Data Injection / 硬件抽象与数据注入

All replay frameworks sometimes require users to use alternative structures that maintain compatibility with replay. AdvantageKit builds all subsystems around [hardware abstraction](#), which provides a clean separation between parts of the code logic that must be isolated: high-level logic, simulation, and replayed code are never able to interact in unintended ways.

所有回放框架有时都会要求用户采用替代结构，以保持与回放的兼容性。AdvantageKit 围绕[硬件抽象](#)构建所有 subsystem，从而清晰分离必须隔离的代码逻辑部分：高层逻辑、仿真和回放代码绝不会以非预期方式相互交互。

The table below compares the implications of this structure against Hoot Replay's approach:

下表将这种结构的影响与 Hoot Replay 的方案进行比较：

	Hardware Abstraction (AdvantageKit) 硬件抽象 (AdvantageKit)	Data Injection (Hoot Replay) 数据注入 (Hoot Replay)
Code Structure 代码结构	The functions of each subsystem are divided into several smaller classes. 每个 subsystem 的功能被划分到几个较小的类中。	All functions of the subsystem are combined into a single large class. subsystem 的所有功能合并在一个大型类中。
Templates 模板	[+] AdvantageKit provides template projects for many subsystems including swerve drives and vision systems (compatible with several vendors). [+] AdvantageKit 为 swerve drive、视觉系统等许多 subsystem 提供 模板项目 （兼容多家厂商）。	[!] Minimal examples are provided. No template projects for subsystems with manual logging. [!] 仅提供极少的示例。没有针对需要手动记录日志的 subsystem 的模板项目。
Data Flow 数据流	[+] Data flow is well-defined to ensure clean separation between real, replay, and sim modes. [+] 数据流有明确的定义，确保真实、回放和仿真模式之间清晰分离。	[-] All data is accessible to all parts of the subsystem. Careful planning and frequent testing is required to ensure that modes are well-separated. [-] subsystem 的所有部分都能访问全部数据。需要仔细规划并频繁测试，才能确保各模式被良好分离。
Input Logging 输入日志记录	[+] Error-free logging of a large number of inputs is facilitated by annotation and record logging. [+] 注解 和 record 日志记录有助于无误地记录大量输入。	[-] Each new input field requires several lines of additional boilerplate, which can easily cause subtle issues during replay if implemented incorrectly. [-] 每增加一个输入字段，都需要额外编写几行样板代码；如果实现不正确，很容易在回放中造成细微问题。
Dashboards Dashboard	[+] Convenience classes are provided to simplify the process of using dashboard inputs . [+] 提供便捷类，以简化使用 dashboard 输入的过程。	[-] All data must be logged manually by the user, even outside of subsystems. [-] 即使在 subsystem 之外，所有数据也都必须由用户手动记录。

Example: Vision Subsystem / 示例：视觉 subsystem

The code below represents a feature-complete Limelight vision subsystem built with both AdvantageKit (hardware abstraction) and Hoot Replay (data injection):

下方代码分别使用 AdvantageKit（硬件抽象）和 Hoot Replay（数据注入），构建了功能完整的 Limelight 视觉 subsystem：

- The AdvantageKit version creates clean separation between the different components of the vision system, making each class easier to understand and debug. The hardware interface with automatic logging enforces clear and correct data flows by default. Annotation, record, and enum logging also allow complex data types to be logged with minimal effort, as shown in the `VisionIOInputs` class below. AdvantageKit 版本清晰分离了视觉系统的不同组成部分，使每个类都更易理解和调试。具有自动日志记录功能的硬件接口 默认 强制保证清晰、正确的数据流。注解、record 和枚举日志记录也使复杂数据类型能以极少的工作量被记录，如下方的 `VisionIOInputs` 类所示。
- The Hoot Replay version combines all of the functionality in a single class, with manual hooks to read and write data for each input field. Note that there is no obvious separation between the the replayed and non-replayed parts of the code, making it easy to read from invalid data sources or replay data incorrectly. The minimal utilities for logging complex types also result in a confusing structure for input data. Hoot Replay 版本将所有功能合并到一个类中，并通过手动添加的钩子读写每个输入字段的数据。请注意，回放部分与非回放部分的代码之间没有明显分离，因此很容易从无效数据源读取数据，或错误地回放数据。记录复杂类型的工具支持非常有限，也使输入数据的结构变得混乱。

AdvantageKit

Subsystem

Vision subsystem (103 lines)

视觉 subsystem (103 行)

```
public class Vision extends SubsystemBase {
    private final VisionConsumer consumer;
    private final VisionIO io;
    private final VisionIOInputsAutoLogged inputs = new VisionIOInputsAutoLogged();
    private final Alert disconnectedAlert =
        new Alert("Vision camera is disconnected.", AlertType.kWarning);

    public Vision(VisionConsumer consumer, VisionIO io) {
        this.consumer = consumer;
        this.io = io;
    }

    /** Returns the X angle to the best target, which can be used for simple servoing with vision. */
    public Rotation2d getTargetX() {
        return inputs.latestTargetObservation.tx();
    }

    @Override
    public void periodic() {
        io.updateInputs(inputs);
        Logger.processInputs("Vision", inputs);

        // Update disconnected alert
        disconnectedAlert.set(!inputs.connected);

        // Initialize logging values
        List<Pose3d> tagPoses = new LinkedList<>();
        List<Pose3d> robotPoses = new LinkedList<>();
        List<Pose3d> robotPosesAccepted = new LinkedList<>();
        List<Pose3d> robotPosesRejected = new LinkedList<>();

        // Add tag poses
        for (int tagId : inputs.tagIds) {
            var tagPose = aprilTagLayout.getTagPose(tagId);
```

```

    if (tagPose.isPresent()) {
        tagPoses.add(tagPose.get());
    }
}

// Loop over pose observations
for (var observation : inputs.poseObservations) {
    // Check whether to reject pose
    boolean rejectPose =
        observation.tagCount() == 0 // Must have at least one tag
        || (observation.tagCount() == 1
            && observation.ambiguity() > maxAmbiguity) // Cannot be high ambiguity
        || Math.abs(observation.pose().getZ()) > maxZError // Must have realistic Z coordinate

    // Must be within the field boundaries
    || observation.pose().getX() < 0.0
    || observation.pose().getX() > aprilTagLayout.getFieldLength()
    || observation.pose().getY() < 0.0
    || observation.pose().getY() > aprilTagLayout.getFieldWidth();

    // Add pose to log
    robotPoses.add(observation.pose());
    if (rejectPose) {
        robotPosesRejected.add(observation.pose());
    } else {
        robotPosesAccepted.add(observation.pose());
    }

    // Skip if rejected
    if (rejectPose) {
        continue;
    }

    // Calculate standard deviations
    double stdDevFactor =
        Math.pow(observation.averageTagDistance(), 2.0) / observation.tagCount();
    double linearStdDev = linearStdDevBaseline * stdDevFactor;
    double angularStdDev = angularStdDevBaseline * stdDevFactor;
    if (observation.type() == PoseObservationType.MEGATAG_2) {
        linearStdDev *= linearStdDevMegatag2Factor;
        angularStdDev *= angularStdDevMegatag2Factor;
    }

    // Send vision observation
    consumer.accept(
        observation.pose().toPose2d(),
        observation.timestamp(),
        VecBuilder.fill(linearStdDev, linearStdDev, angularStdDev));
}

// Log camera metadata
Logger.recordOutput("Vision/TagPoses", tagPoses.toArray(new Pose3d[0]));
Logger.recordOutput("Vision/RobotPoses", robotPoses.toArray(new Pose3d[0]));
Logger.recordOutput("Vision/RobotPosesAccepted", robotPosesAccepted.toArray(new Pose3d[0]));
Logger.recordOutput("Vision/RobotPosesRejected", robotPosesRejected.toArray(new Pose3d[0]));
}

```

```
@FunctionalInterface
public static interface VisionConsumer {
    public void accept(
        Pose2d visionRobotPoseMeters,
        double timestampSeconds,
        Matrix<N3, N1> visionMeasurementStdDevs);
}
}
```

Hardware Interface / 硬件接口

Vision hardware interface (30 lines)

视觉硬件接口 (30 行)

```
public interface VisionIO {
    @AutoLog
    public static class VisionIOInputs {
        public boolean connected = false;
        public TargetObservation latestTargetObservation =
            new TargetObservation(Rotation2d.kZero, Rotation2d.kZero);
        public PoseObservation[] poseObservations = new PoseObservation[0];
        public int[] tagIds = new int[0];
    }

    /** Represents the angle to a simple target, not used for pose estimation. */
    public static record TargetObservation(Rotation2d tx, Rotation2d ty) {}

    /** Represents a robot pose sample used for pose estimation. */
    public static record PoseObservation(
        double timestamp,
        Pose3d pose,
        double ambiguity,
        int tagCount,
        double averageTagDistance,
        PoseObservationType type) {}

    public static enum PoseObservationType {
        MEGATAG_1,
        MEGATAG_2,
        PHOTONVISION
    }

    public default void updateInputs(VisionIOInputs inputs) {}
}
```

Hardware Implementation / 硬件实现

Vision hardware implementation (103 lines)

视觉硬件实现 (103 行)

```
public class VisionIOLimelight implements VisionIO {
    private final Supplier<Rotation2d> rotationSupplier;
    private final DoubleArrayPublisher orientationPublisher;

    private final DoubleSubscriber latencySubscriber;
    private final DoubleSubscriber txSubscriber;
    private final DoubleSubscriber tySubscriber;
    private final DoubleArraySubscriber megatag1Subscriber;
```

```

private final DoubleArraySubscriber megatag2Subscriber;

/**
 * Creates a new VisionIOLimelight.
 *
 * @param name The configured name of the Limelight.
 * @param rotationSupplier Supplier for the current estimated rotation, used for MegaTag 2.
 */
public VisionIOLimelight(String name, Supplier<Rotation2d> rotationSupplier) {
    var table = NetworkTableInstance.getDefault().getTable(name);
    this.rotationSupplier = rotationSupplier;
    orientationPublisher = table.getDoubleArrayTopic("robot_orientation_set").publish();
    latencySubscriber = table.getDoubleTopic("t1").subscribe(0.0);
    txSubscriber = table.getDoubleTopic("tx").subscribe(0.0);
    tySubscriber = table.getDoubleTopic("ty").subscribe(0.0);
    megatag1Subscriber = table.getDoubleArrayTopic("botpose_wpiblu").subscribe(new double[] {});
    megatag2Subscriber =
        table.getDoubleArrayTopic("botpose_orb_wpiblu").subscribe(new double[] {});
}

@Override
public void updateInputs(VisionIOInputs inputs) {
    // Update connection status based on whether an update has been seen in the last 250ms
    inputs.connected =
        ((RobotController.getFPGATime() - latencySubscriber.getLastChange()) / 1000) < 250;

    // Update target observation
    inputs.latestTargetObservation =
        new TargetObservation(
            Rotation2d.fromDegrees(txSubscriber.get()), Rotation2d.fromDegrees(tySubscriber.get()));

    // Update orientation for MegaTag 2
    orientationPublisher.accept(
        new double[] {rotationSupplier.get().getDegrees(), 0.0, 0.0, 0.0, 0.0, 0.0});
    NetworkTableInstance.getDefault()
        .flush(); // Increases network traffic but recommended by Limelight

    // Read new pose observations from NetworkTables
    Set<Integer> tagIds = new HashSet<>();
    List<PoseObservation> poseObservations = new LinkedList<>();
    for (var rawSample : megatag1Subscriber.readQueue()) {
        if (rawSample.value.length == 0) continue;
        for (int i = 11; i < rawSample.value.length; i += 7) {
            tagIds.add((int) rawSample.value[i]);
        }
        poseObservations.add(
            new PoseObservation(
                rawSample.timestamp * 1.0e-6 - rawSample.value[6] * 1.0e-3,
                parsePose(rawSample.value),
                rawSample.value.length >= 18 ? rawSample.value[17] : 0.0,
                (int) rawSample.value[7],
                rawSample.value[9],
                PoseObservationType.MEGATAG_1));
    }
    for (var rawSample : megatag2Subscriber.readQueue()) {
        if (rawSample.value.length == 0) continue;
        for (int i = 11; i < rawSample.value.length; i += 7) {
            tagIds.add((int) rawSample.value[i]);
        }
        poseObservations.add(
            new PoseObservation(
                rawSample.timestamp * 1.0e-6 - rawSample.value[6] * 1.0e-3,
                parsePose(rawSample.value),
                0.0,

```

```

        (int) rawSample.value[7],
        rawSample.value[9],
        PoseObservationType.MEGATAG_2));
    }

    // Save pose observations to inputs object
    inputs.poseObservations = new PoseObservation[poseObservations.size()];
    for (int i = 0; i < poseObservations.size(); i++) {
        inputs.poseObservations[i] = poseObservations.get(i);
    }

    // Save tag IDs to inputs objects
    inputs.tagIds = new int[tagIds.size()];
    int i = 0;
    for (int id : tagIds) {
        inputs.tagIds[i++] = id;
    }
}

/** Parses the 3D pose from a Limelight botpose array. */
private static Pose3d parsePose(double[] rawLLArray) {
    return new Pose3d(
        rawLLArray[0],
        rawLLArray[1],
        rawLLArray[2],
        new Rotation3d(
            Units.degreesToRadians(rawLLArray[3]),
            Units.degreesToRadians(rawLLArray[4]),
            Units.degreesToRadians(rawLLArray[5])));
}
}

```

Hoot Replay

Vision subsystem and hardware interface (248 lines)

视觉 subsystem 与硬件接口 (248 行)

```

public class HootVision extends SubsystemBase {
    private final VisionConsumer consumer;
    private final Alert disconnectedAlert =
        new Alert("Vision camera is disconnected.", AlertType.kWarning);
    private final Supplier<Rotation2d> rotationSupplier;
    private final DoubleArrayPublisher orientationPublisher;

    private final DoubleSubscriber latencySubscriber;
    private final DoubleSubscriber txSubscriber;
    private final DoubleSubscriber tySubscriber;
    private final DoubleArraySubscriber megatag1Subscriber;
    private final DoubleArraySubscriber megatag2Subscriber;

    private boolean connected = false;
    private Rotation2d latestTargetObservationTx = Rotation2d.kZero;
    private Rotation2d latestTargetObservationTy = Rotation2d.kZero;
    private double[] timestamps = new double[] {};
    private Pose3d[] poses = new Pose3d[] {};
    private double[] ambiguities = new double[] {};
    private int[] tagCounts = new int[] {};
    private double[] averageTagDistances = new double[] {};
    private int[] types = new int[] {};
    public int[] tagIds = new int[] {};
}

```

```

private final HootAutoReplay hootReplay =
    new HootAutoReplay()
        .withBoolean("Connected", () -> connected, (value) -> connected = value.value)
        .withStruct(
            "Vision/LatestTargetObservationTx",
            Rotation2d.struct,
            () -> latestTargetObservationTx,
            (value) -> latestTargetObservationTx = value.value)
        .withStruct(
            "Vision/LatestTargetObservationTy",
            Rotation2d.struct,
            () -> latestTargetObservationTy,
            (value) -> latestTargetObservationTy = value.value)
        .withDoubleArray(
            "Vision/Timestamps", () -> timestamps, (value) -> timestamps = value.value)
        .withStructArray(
            "Vision/Poses", Pose3d.struct, () -> poses, (value) -> poses = value.value)
        .withDoubleArray(
            "Vision/Ambiguities", () -> ambiguities, (value) -> ambiguities = value.value)
        .withIntegerArray(
            "Vision/Timestamps",
            () -> Arrays.stream(tagCounts).mapToLong(i -> i).toArray(),
            (value) -> tagCounts = Arrays.stream(value.value).mapToInt(i -> (int) i).toArray())
        .withDoubleArray(
            "Vision/AverageTagDistances",
            () -> averageTagDistances,
            (value) -> averageTagDistances = value.value)
        .withIntegerArray(
            "Vision/Types",
            () -> Arrays.stream(types).mapToLong(i -> i).toArray(),
            (value) -> types = Arrays.stream(value.value).mapToInt(i -> (int) i).toArray())
        .withIntegerArray(
            "Vision/TagIds",
            () -> Arrays.stream(tagIds).mapToLong(i -> i).toArray(),
            (value) -> tagIds = Arrays.stream(value.value).mapToInt(i -> (int) i).toArray());

public HootVision(VisionConsumer consumer, String name, Supplier<Rotation2d> rotationSupplier) {
    this.consumer = consumer;
    var table = NetworkTableInstance.getDefault().getTable(name);
    this.rotationSupplier = rotationSupplier;
    orientationPublisher = table.getDoubleArrayTopic("robot_orientation_set").publish();
    latencySubscriber = table.getDoubleTopic("t1").subscribe(0.0);
    txSubscriber = table.getDoubleTopic("tx").subscribe(0.0);
    tySubscriber = table.getDoubleTopic("ty").subscribe(0.0);
    megatag1Subscriber = table.getDoubleArrayTopic("botpose_wpiblue").subscribe(new double[] {});
    megatag2Subscriber =
        table.getDoubleArrayTopic("botpose_orb_wpiblue").subscribe(new double[] {});
}

@Override
public void periodic() {
    if (!Utils.isReplay()) {
        // Update connection status based on whether an update has been seen in the last 250ms
        connected =
            ((RobotController.getFPGATime() - latencySubscriber.getLastChange()) / 1000) < 250;

        // Update target observation
        latestTargetObservationTx = Rotation2d.fromDegrees(txSubscriber.get());
        latestTargetObservationTy = Rotation2d.fromDegrees(tySubscriber.get());

        // Update orientation for MegaTag 2
        orientationPublisher.accept(
            new double[] {rotationSupplier.get().getDegrees(), 0.0, 0.0, 0.0, 0.0, 0.0});
        NetworkTableInstance.getDefault()
    }
}

```

```

        .flush(); // Increases network traffic but recommended by Limelight

// Read new pose observations from NetworkTables
Set<Integer> tagIds = new HashSet<>();
List<PoseObservation> poseObservations = new LinkedList<>();
for (var rawSample : megatag1Subscriber.readQueue()) {
    if (rawSample.value.length == 0) continue;
    for (int i = 11; i < rawSample.value.length; i += 7) {
        tagIds.add((int) rawSample.value[i]);
    }
    poseObservations.add(
        new PoseObservation(
            rawSample.timestamp * 1.0e-6 - rawSample.value[6] * 1.0e-3,
            parsePose(rawSample.value),
            rawSample.value.length >= 18 ? rawSample.value[17] : 0.0,
            (int) rawSample.value[7],
            rawSample.value[9],
            PoseObservationType.MEGATAG_1));
}
for (var rawSample : megatag2Subscriber.readQueue()) {
    if (rawSample.value.length == 0) continue;
    for (int i = 11; i < rawSample.value.length; i += 7) {
        tagIds.add((int) rawSample.value[i]);
    }
    poseObservations.add(
        new PoseObservation(
            rawSample.timestamp * 1.0e-6 - rawSample.value[6] * 1.0e-3,
            parsePose(rawSample.value),
            0.0,
            (int) rawSample.value[7],
            rawSample.value[9],
            PoseObservationType.MEGATAG_2));
}

// Save pose observations to inputs
timestamps = new double[poseObservations.size()];
poses = new Pose3d[poseObservations.size()];
ambiguities = new double[poseObservations.size()];
tagCounts = new int[poseObservations.size()];
averageTagDistances = new double[poseObservations.size()];
types = new int[poseObservations.size()];
for (int i = 0; i < poseObservations.size(); i++) {
    var obs = poseObservations.get(i);
    timestamps[i] = obs.timestamp();
    poses[i] = obs.pose();
    ambiguities[i] = obs.ambiguity();
    tagCounts[i] = obs.tagCount();
    averageTagDistances[i] = obs.averageTagDistance();
    types[i] = obs.type().ordinal();
}

// Save tag IDs to inputs
this.tagIds = new int[tagIds.size()];
int i = 0;
for (int tagId : tagIds) {
    this.tagIds[i++] = tagId;
}
}
hootReplay.update();

// Update disconnected alert
disconnectedAlert.set(!connected);

// Initialize logging values

```

```

List<Pose3d> tagPoses = new LinkedList<>();
List<Pose3d> robotPoses = new LinkedList<>();
List<Pose3d> robotPosesAccepted = new LinkedList<>();
List<Pose3d> robotPosesRejected = new LinkedList<>();

// Add tag poses
for (int tagId : tagIds) {
    var tagPose = aprilTagLayout.getTagPose(tagId);
    if (tagPose.isPresent()) {
        tagPoses.add(tagPose.get());
    }
}

// Loop over pose observations
for (int i = 0; i < timestamps.length; i++) {
    // Check whether to reject pose
    boolean rejectPose =
        tagCounts[i] == 0 // Must have at least one tag
        || (tagCounts[i] == 1 && ambiguities[i] > maxAmbiguity) // Cannot be high ambiguity
        || Math.abs(poses[i].getZ()) > maxZError // Must have realistic Z coordinate

        // Must be within the field boundaries
        || poses[i].getX() < 0.0
        || poses[i].getX() > aprilTagLayout.getFieldLength()
        || poses[i].getY() < 0.0
        || poses[i].getY() > aprilTagLayout.getFieldWidth();

    // Add pose to log
    robotPoses.add(poses[i]);
    if (rejectPose) {
        robotPosesRejected.add(poses[i]);
    } else {
        robotPosesAccepted.add(poses[i]);
    }

    // Skip if rejected
    if (rejectPose) {
        continue;
    }

    // Calculate standard deviations
    double stdDevFactor = Math.pow(averageTagDistances[i], 2.0) / tagCounts[i];
    double linearStdDev = linearStdDevBaseline * stdDevFactor;
    double angularStdDev = angularStdDevBaseline * stdDevFactor;
    if (types[i] == 1) {
        linearStdDev *= linearStdDevMegatag2Factor;
        angularStdDev *= angularStdDevMegatag2Factor;
    }

    // Send vision observation
    consumer.accept(
        poses[i].toPose2d(),
        timestamps[i],
        VecBuilder.fill(linearStdDev, linearStdDev, angularStdDev));
}

// Log camera metadata
SignalLogger.writeStructArray(
    "Vision/TagPoses", Pose3d.struct, tagPoses.toArray(new Pose3d[0]));
SignalLogger.writeStructArray(
    "Vision/RobotPoses", Pose3d.struct, robotPoses.toArray(new Pose3d[0]));
SignalLogger.writeStructArray(
    "Vision/RobotPosesAccepted",
    Pose3d.struct,

```

```

        robotPosesAccepted.toArray(new Pose3d[0]));
    SignalLogger.writeStructArray(
        "Vision/RobotPosesRejected",
        Pose3d.struct,
        robotPosesRejected.toArray(new Pose3d[0]));
}

/** Returns the X angle to the best target, which can be used for simple servoing with vision. */
public Rotation2d getTargetX() {
    return Rotation2d.fromDegrees(txSubscriber.get());
}

/** Parses the 3D pose from a Limelight botpose array. */
private static Pose3d parsePose(double[] rawLLArray) {
    return new Pose3d(
        rawLLArray[0],
        rawLLArray[1],
        rawLLArray[2],
        new Rotation3d(
            Units.degreesToRadians(rawLLArray[3]),
            Units.degreesToRadians(rawLLArray[4]),
            Units.degreesToRadians(rawLLArray[5])));
}

@FunctionalInterface
public static interface VisionConsumer {
    public void accept(
        Pose2d visionRobotPoseMeters,
        double timestampSeconds,
        Matrix<N3, N1> visionMeasurementStdDevs);
}
}

```

DANGER / 危险

Did you notice that this example of Hoot Replay actually has **three separate** subtle but critical issues that prevent replay from functioning? The monolithic structure of data injection and a lack of automatic logging options make subtle typos extremely common and challenging to debug.

你是否注意到，这个 Hoot Replay 示例实际上存在**三个彼此独立**、细微但关键的问题，导致回放无法工作？数据注入的单体结构，以及缺乏自动日志记录选项，使细微的笔误极为常见且难以调试。

Miscellaneous / 其他事项

The table below provides an overview of the differences between each replay tool.

下表概述了各个回放工具之间的差异。

	AdvantageKit	Hoot Replay
Accuracy	[+] Deterministic	[-] Non-deterministic
准确性	[+] 确定性	[-] 非确定性
Rapid Iteration	[+] Replay at any speed	[-] Accuracy decreases with speed
快速迭代	[+] 以任意速度回放	[-] 准确性随速度提升而下降
Code Structure	[+] Hardware abstraction + automatic logging	[-] Manual data injection
代码结构	[+] 硬件抽象 + 自动日志记录	[-] 手动数据注入
Vendor	[+] No restriction + templates for multiple vendors	[-] Locked to CTRE devices
厂商	[+] 无限制 + 多家厂商的模板	[-] 仅限 CTRE 设备
FRC Languages	Java	Java
FRC 语言		
Pricing	Free & Open Source	[Fee] Subscription: Requires Phoenix Pro
价格	免费且开源	[Fee] 订阅: 需要 Phoenix Pro
Users in 2026	852 teams	<10 teams
2026 年用户	852 支队伍	<10 支队伍

NOTE / 说明

The number of AdvantageKit users is based on official usage reporting data published by FIRST. The number of Hoot Replay users is estimated based on a search of public GitHub repositories adjusted based on the percentage of all teams that publish code on GitHub.

AdvantageKit 的用户数量基于 FIRST 发布的官方使用情况报告数据。Hoot Replay 的用户数量通过搜索公开的 GitHub 仓库估算，并根据在 GitHub 上公开代码的队伍占全部队伍的比例进行调整。

High-Frequency Odometry / 高频 odometry

The AdvantageKit swerve templates support high-frequency odometry on both Spark and TalonFX(S) hardware, which means that data from the drive motors, encoders, and gyro are sampled faster than the primary 50 Hz loop cycle. The purpose of high-frequency odometry is to improve the accuracy and consistency of odometry data.

AdvantageKit 的 swerve 模板在 Spark 和 TalonFX(S) 硬件上都支持高频里程计 (odometry)，即以快于主 50 Hz 循环周期的频率对底盘电机、编码器和陀螺仪的数据进行采样。高频 odometry 的目的是提高 odometry 数据的准确性和一致性。

The testing below was conducted by Team 6328 in November 2023 to measure the benefit of high-frequency odometry compared to traditional 50 Hz odometry. Note that this data is broadly applicable to any application of this technique, not just the AdvantageKit templates. This testing builds on the work done by CTRE [here](#), which was performed under inconsistent testing conditions with a very small sample size.

以下测试由 6328 队于 2023 年 11 月进行，用于衡量高频 odometry 相较于传统 50 Hz odometry 的收益。请注意，这些数据普遍适用于该技术的各种应用，并不局限于 AdvantageKit 模板。此测试建立在 CTRE [此处](#)工作的基础上；CTRE 的测试条件不一致，且样本量非常小。

Setup / 测试设置

We focused our testing on the accuracy of odometry during an autonomous path with no vision assistance. This is where the accuracy of wheel odometry is most critical - during teleop, additional sensors like vision will always be necessary to maintain accuracy.

我们的测试重点是，在没有视觉辅助的自动路径中，odometry 的准确性。这是轮式 odometry 准确性最关键的场景；在 teleop 期间，始终需要视觉等额外传感器来维持准确性。

Using a NEO-based swerve, we ran a ~12 second PathPlanner auto as shown below. This is intended to be representative of the type of movement that might be seen during an auto path, including holonomic rotations, movement in multiple directions, and occasional hard accelerations.

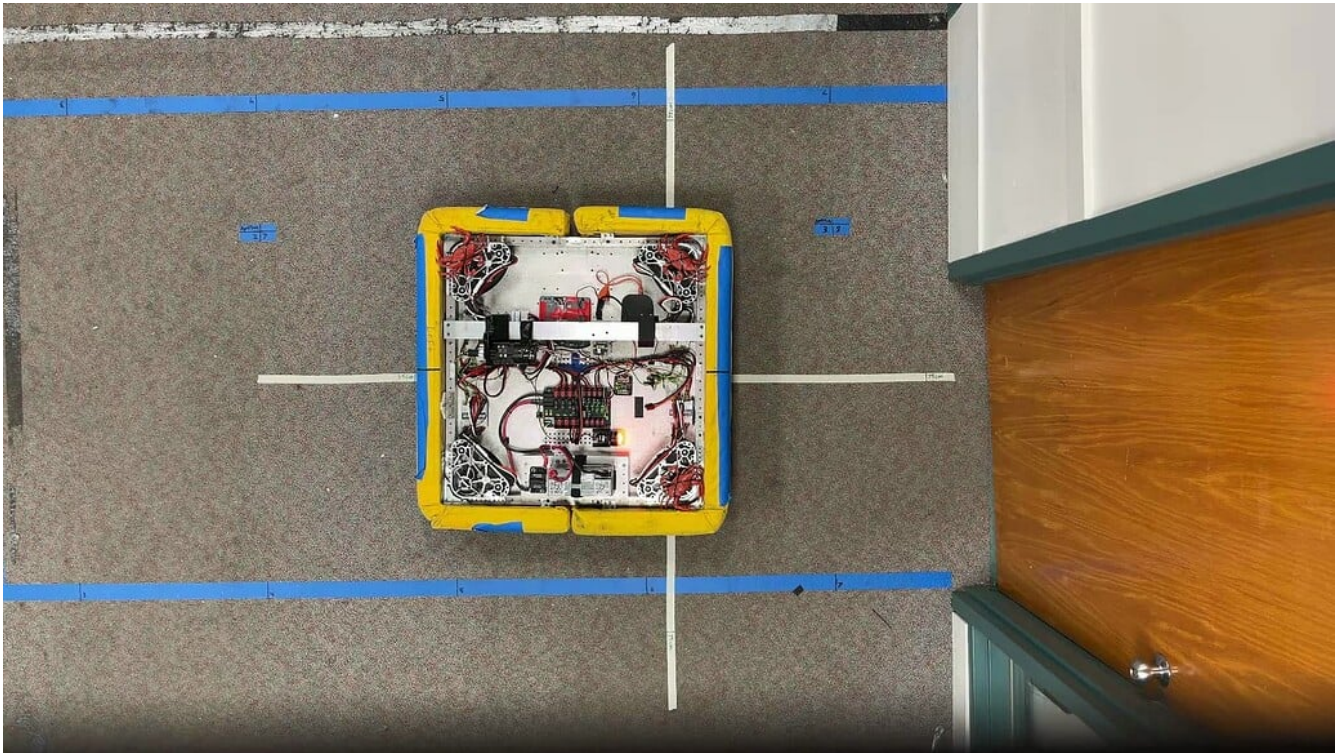
使用基于 NEO 的 swerve，我们运行了一个约 12 秒的 PathPlanner 自动程序，如下所示。它旨在代表自动路径中可能出现的运动类型，包括全向运动中的旋转、多方向移动以及偶尔出现的剧烈加速。



Test path / 测试路径

The start location of the auto is marked with the large white cross, and the robot was commanded to return to the same position. Log data shows that the robot's odometry always reached the target with negligible error. Using a camera positioned in the ceiling, we are able to estimate the robot's true ending position based on a tape marker at the center of the robot (example shown below). Any error from the target position to the robot's position represents odometry error that was accumulated during the path.

自动程序的起始位置以较大的白色十字标记，机器人被指令返回同一位置。日志数据显示，机器人的 odometry 始终以可忽略的误差到达目标。通过安装在天花板上的相机，我们可以根据机器人中心的胶带标记，估算机器人的实际结束位置（示例如下）。从目标位置到机器人位置的任何误差，都代表沿路径累积的 odometry 误差。

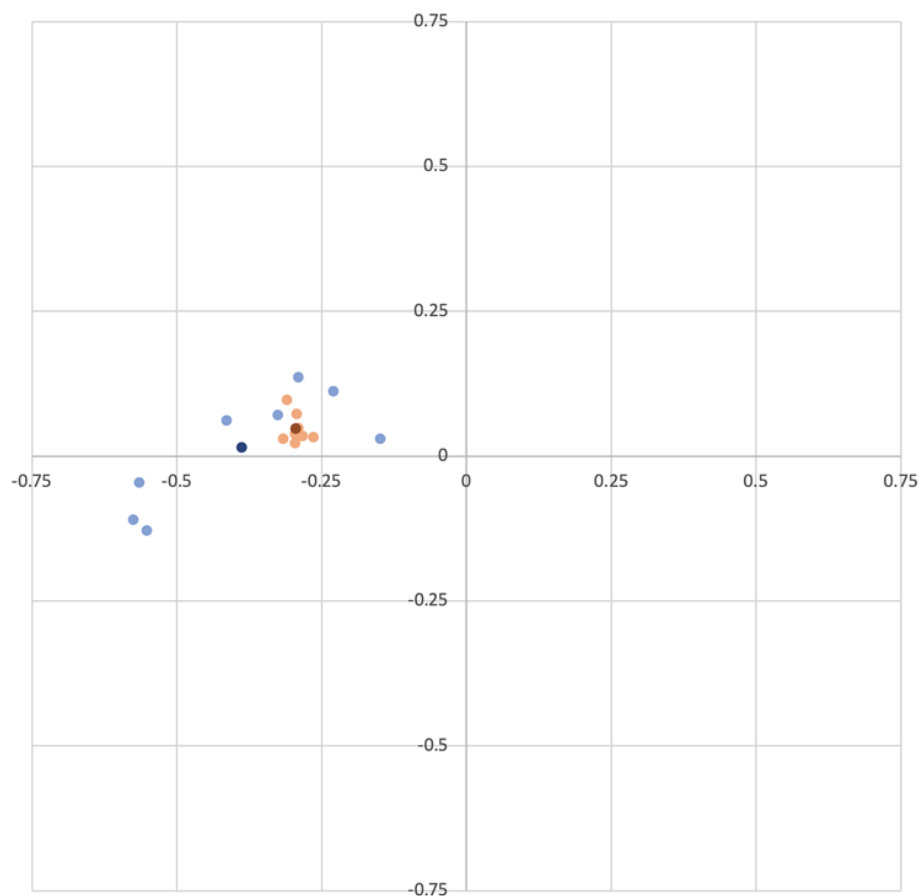


Example measurement / 测量示例

Results / 结果

We repeated this auto 16 times - 8 times with 50Hz odometry and 8 times with 250Hz odometry. The ending positions are plotted below in meters, where blue is 50Hz and orange in 250Hz. The dark circles represent the average ending position of each set of samples.

我们将此自动程序重复运行了 16 次：8 次使用 50Hz odometry，8 次使用 250Hz odometry。下图以米为单位绘制了结束位置，其中蓝色表示 50Hz，橙色表示 250Hz。深色圆点代表每组样本的平均结束位置。



Results / 结果

The mean error and standard deviations are shown below. Note that the improvement in mean error was relatively small compared to the improvement in standard deviation. In practice, this means that **autos are unlikely to be more accurate, but they will be much more consistent/precise.**

下表显示平均误差和标准差。请注意，相较于标准差的改善，平均误差的改善幅度相对较小。在实际中，这意味着**自动程序不太可能更准确，但其一致性/精密度会显著提高。**

	Mean Error (m) 平均误差 (m)	Standard Deviation (m) 标准差 (m)
50Hz	0.388	0.180
250Hz	0.297	0.028
% Improvement % 改善幅度	23.3%	84.4%

CANivore Timesync / CANivore Timesync

The TalonFX(S) swerve template for AdvantageKit supports [CANivore Timesync](#) for Phoenix Pro subscribers running the drive on a [CANivore](#). CTRE conducted testing on the impact of timesync on odometry accuracy, which can be found [here](#). This part of CTRE's testing was based on a more reliable testing methodology than the rest of the linked page, using autonomous driving similar to the approach described here.

对于订阅了 Phoenix Pro 并在 [CANivore](#) 上运行底盘的用户，AdvantageKit 的 TalonFX(S) swerve 模板支持 [CANivore Timesync](#)。CTRE 测试了 timesync 对 odometry 准确性的影响，结果见[此处](#)。与所链接页面中的其他测试相比，CTRE 的这部分测试采用了更可靠的方法，使用的自动行驶方式与本文描述的方法类似。

They found an improvement in error of ~15% and in standard deviation of ~34%. This is a measurable but relatively modest improvement compared to high-frequency odometry, and similarly has a more significant impact on consistency (standard deviation) than overall accuracy (error).

他们发现，误差改善了约 15%，标准差改善了约 34%。相较于高频 odometry，这是一项可测量但相对较小的改善；同样，它对一致性（标准差）的影响比对整体准确性（误差）的影响更显著。

Deterministic Timestamps / 确定性时间戳

The Problem / 问题

To guarantee accurate replay, all of the data used by the robot code must be included in the log file and replayed in simulation. This includes the current timestamp, which may be used for calculations in control loops. WPILib's default behavior is to read the timestamp directly from the FPGA every time a method like `Timer.getTimestamp()` is called. Every return value will be slightly different as the timestamp continues increasing throughout each loop cycle. This behavior is problematic for AdvantageKit replay because the return values from those method calls are not logged and **cannot be accurately replayed in simulation**.

为保证准确回放，机器人代码使用的全部数据都必须包含在日志文件中，并在仿真中回放。这包括可能用于控制循环计算的当前时间戳。WPILib 的默认行为是，每次调用 `Timer.getTimestamp()` 等方法时都直接从 FPGA 读取时间戳。由于时间戳在每个循环周期内持续增加，每次返回值都会略有不同。这种行为会给 AdvantageKit 回放带来问题，因为这些方法调用的返回值没有被记录，**无法在仿真中准确回放**。

AdvantageKit's solution is to use synchronized timestamps. The timestamp is read once from the FPGA at the start of the loop cycle and injected into WPILib. By default, all calls to `Timer.getTimestamp()` or similar return the synchronized timestamp from AdvantageKit instead of the "real" FPGA time. During replay, the logged timestamp is used for each loop cycle. The result is that all of the control logic is deterministic and will **exactly match the behavior of the real robot**.

AdvantageKit 的解决方案是使用 同步时间戳。每个循环周期开始时，从 FPGA 读取一次时间戳，并将其注入 WPILib。默认情况下，对 `Timer.getTimestamp()` 或类似方法的所有调用，都返回 AdvantageKit 的同步时间戳，而不是“真实”的 FPGA 时间。回放期间，每个循环周期都使用记录的时间戳。因此，所有控制逻辑都具有 确定性，并且会**与真实机器人的行为完全一致**。

Solution #1 / 解决方案 #1

Where precise timestamps are required, the best solution is to record measurement timestamps as part of the input data for each subsystem. For example, [NetworkTables](#) and [Phoenix 6](#) include methods to read the timestamp of each update, which can be used in calculations for wheel odometry, vision localization, or other precise controls alongside AdvantageKit's deterministic timestamps. For most use cases, measuring the timestamp of the original sample is more desirable than measuring the precise timestamp on the robot.

需要精确时间戳时，最佳方案是将测量时间戳作为每个 subsystem 输入数据的一部分进行记录。例如，[NetworkTables](#) 和 [Phoenix 6](#) 都提供了读取每次更新的时间戳的方法；这些时间戳可以与 AdvantageKit 的确定性时间戳一起，用于轮式 odometry、视觉定位或其他精密控制的计算。对于大多数用途，测量原始采样的时间戳比测量机器人上的精确时间戳更理想。

Solution #2 / 解决方案 #2

`Timer.getMonotonicTimestamp()` can always be used to access the "real" FPGA timestamp where necessary, like within IO implementations or for analyzing performance. This method is not affected by log replay (i.e. it will not reflect the accelerated rate of replay). One use case for this method is measuring code execution time, since those values don't need to be recreated during log replay. WPILib classes like Watchdog use this method because they use the timestamp for analyzing performance and are not part of the robot's control logic.

在 IO 实现内部或分析性能等确有需要的场景中，始终可以使用 `Timer.getMonotonicTimestamp()` 访问“真实”的 FPGA 时间戳。此方法不受日志回放影响（即不会反映加速后的回放速率）。它的一种用途是测量代码执行时间，因为这些值无需在日志回放中重现。Watchdog 等 WPILib 类使用此方法，因为它们使用时间戳来分析性能，并不属于机器人的控制逻辑。

Solution #3 / 解决方案 #3

Optionally, AdvantageKit allows you to disable deterministic timestamps. This reverts to the default WPILib behavior of reading from the FPGA for every method call, making the behavior of the robot code non-deterministic. The "output" values seen in simulation may be slightly different than they were on the real robot. This alternative mode should only be used when all of the following are true:

AdvantageKit 还允许选择禁用确定性时间戳。这会恢复 WPILib 每次方法调用都从 FPGA 读取的默认行为，使机器人代码的行为变为非确定性的。仿真中看到的“输出”值可能与真实机器人上的值略有不同。只有在以下条件全部成立时，才应使用这种替代模式：

1. The control logic depends on the exact timestamp within a single loop cycle, like a high precision control loop that is significantly affected by the precise time that it is executed within each (usually 20ms) loop cycle.

控制逻辑依赖单个循环周期内部的精确时间戳，例如高精度控制循环，它会显著受到其在每个循环周期（通常为 20ms）内具体执行时刻的影响。

2. The sensor values used in the loop cannot be associated with timestamps in an IO implementation. See solution #1.

循环中使用的传感器值无法在 IO 实现中与时间戳关联。参见解决方案 #1。

3. The IO (sensors, actuators, etc) involved in the loop are sufficiently low-latency that the exact timestamp on the RIO is significant. For example, CAN motor controllers are limited by the rate of their CAN frames, so the extra precision on the RIO is insignificant in most cases.

循环涉及的 IO（传感器、执行器等）延迟足够低，因此 RIO 上的精确时间戳具有实际意义。例如，CAN 电机控制器受 CAN 帧速率限制，所以在大多数情况下，RIO 上额外的精度并无实际意义。

If you need to disable deterministic timestamps globally, add the following lines to the constructor of Robot after `Logger.start()`:

如果需要全局禁用确定性时间戳，请在 Robot 构造函数中、`Logger.start()` 之后添加以下代码：

```
if (!Logger.hasReplaySource()) {
    RobotController.setTimeSource(RobotController::getMonotonicTime);
}
```

Replay Case Studies / 回放案例

Elevator Profile / 升降机构运动曲线

FRC Log Replay and Simulation (2025) - FRC 6328 FIRST Championship Conference

FRC 日志回放与仿真 (2025) - FRC 6328 FIRST Championship 技术讲座

Autoscoreing / 自动得分

FRC Log Replay and Simulation (2025) - FRC 6328 FIRST Championship Conference

FRC 日志回放与仿真 (2025) - FRC 6328 FIRST Championship 技术讲座

Command Gremlins / Command 中的隐蔽问题

FRC Log Replay and Simulation (2025) - FRC 6328 FIRST Championship Conference

FRC 日志回放与仿真 (2025) - FRC 6328 FIRST Championship 技术讲座

Aiming Functions / 瞄准函数

FRC Log Replay and Simulation (2025) - FRC 6328 FIRST Championship Conference

FRC 日志回放与仿真 (2025) - FRC 6328 FIRST Championship 技术讲座

AprilTag Vision / AprilTag 视觉

FRC Log Replay and Simulation (2025) - FRC 6328 FIRST Championship Conference

FRC 日志回放与仿真 (2025) - FRC 6328 FIRST Championship 技术讲座

Traditional Vision / 传统视觉

FRC Log Replay and Simulation (2024) - FRC 6328 FIRST Championship Conference

FRC 日志回放与仿真 (2024) - FRC 6328 FIRST Championship 技术讲座